

TP4

Introduction au C++

Programmation Orientée Objet

Exercice 1: Exercice (16 points)

Soit le programme suivant :

```
void main() {
    CBasePile* ptPile ;
    CPileFifo fifo ; // pile First In First Out
    CPileLifo lifo ; // pile Last In First Out

    ptPile = &fifo;          // ptPile pointe vers la pile FIFO
    *ptPile < 0 < 1;          // on empile 0 puis 1
    cout << "pile=" << *ptPile ; // a l'ecran : pile=0 1

    int i;
    *ptPile > i ;              // depile dans i
    cout << "pile=" << *ptPile << " i=" << i ;
                                // a l'ecran : pile=1 i=0

    ptPile = &lifo ;           // ptPile pointe vers la pile LIFO
    *ptPile < 0 < 1 ;          // on empile 0 puis 1
    cout << "pile=" << *ptPile ; // a l'ecran : pile=0 1

    *ptPile > i ;              // depile dans i
    cout << "pile=" << *ptPile << " i=" << i ;
                                // a l'ecran : pile=0 i=1
}
```

Le but de cet exercice est de regrouper dans une classe de base, CBasePile, les opérations communes aux piles LIFO (Last In First Out) et FIFO (First In First Out), puis de spécialiser cette classe par dérivation en une classe permettant de gérer une pile LIFO, et une classe permettant de gérer une pile FIFO. Pour cela, il faut répondre aux questions suivantes :

1. Quelle structure de données peut être commune à une pile LIFO et FIFO ? En déduire les données membres de la classe de base.
2. D'après le programme ci-dessus, quels sont les opérateurs communs aux piles LIFO et FIFO ? En déduire les fonctions membres de la classe de base.
3. La classe de base contient les données membres communes à une pile LIFO et FIFO, mais c'est dans les classes dérivées que doivent être spécialisées les fonctions membres. Écrire alors les classes dérivées CPileLifo et CPileFifo pour les piles LIFO et FIFO respectivement.
4. Comment assurer la ligature dynamique des fonctions pour faire fonctionner le programme ci-dessus (appel des fonctions des classes dérivées via un pointeur sur une classe de base) ?

Exercice 2: Gestion mémoire (4 points)

Dans l'exercice précédent, le tableau donnees_ de CBasePile est alloué dynamiquement avec `new int[capacite_]` et doit être libéré par le destructeur avec `delete[]`.

1. Le code suivant contient une fuite mémoire. Identifier la ligne responsable et corriger le code.

```
void foo() {
    CPileFifo* p = new CPileFifo(5) ; // alloue sur le tas
```

```
*p < 0 < 1 ;  
// ... utilisation de p ...  
} // fin de foo
```

2. On crée une pile via un pointeur de classe de base et on la détruit ainsi :

```
CBasePile* ptPile = new CPileFifo(5) ;  
// ... utilisation ...  
delete ptPile ;
```

Pourquoi le destructeur de `CBasePile` doit-il être déclaré `virtual` ? Que se passe-t-il si ce mot-clé est absent ?